

PRODUCT REQUIREMENT DOCUMENT

Tiktok Shop API Integration & Fulfillment Platform

A Comprehensive Warehouse Packing, Order Synchronization, and Shipping Manifest Automation Engine.

APPLICATION CREDENTIALS & SERVICE PROFILE

Partner App Name:	Tiktok Fulfillment Platform
Developer Entity:	HSG Global Pte Ltd
TikTok Service ID:	7665903523104081672
OAuth Redirect URI:	https://ib.hsgglobalpteltd.workers.dev/api/tiktok/auth/callback
Target Scope Packages:	Authorization, Order Management, Fulfillment
Deployment Stack:	Next.js 16 + Cloudflare Workers + Supabase SQL + R2 Storage
Document Version:	1.0.0 (Release - Production)
Publish Date:	August 2026

Table of Contents

1. Executive Summary & Business Scenario
2. Key Platform Features & Operating Workflows
3. System Architecture & Integrated Data Flows
4. Technical API Endpoints & Request Helpers Reference
5. TikTok Shop Open Platform Scopes Justification

1. Executive Summary & Business Scenario

HSG Global Pte Ltd is a major regional physical goods distributor and e-commerce seller. Operating multiple high-volume storefronts, the warehouse handles thousands of parcels daily. Traditional logistics rely on manual picking sheets, leading to significant packaging errors (incorrect item variations, wrong quantities, missing items, or mismatched shipping labels). These errors lead to negative customer reviews, buyer disputes, and TikTok seller penalty points.

The TikTok Fulfillment Platform (Project 4) was designed specifically as a warehouse-terminal application to resolve these challenges. Directly connecting to the TikTok Shop Open API, it bridges digital orders with physical fulfillment workflows.

Key Business Objectives:

- Ensure zero-error picking and packing by forcing barcode scans on every item.
- Automate label generation and instant thermal printing at packaging stations.
- Capture visual quality-control records (photos) for before-packing (open bin) and after-packing (sealed boxes).
- Integrate AI vision models (Gemini Flash API) to ensure shipping labels are physically attached to the package prior to ship scheduling, preventing compliance bypasses by operators.

2. Key Platform Features & Operating Workflows

2.1 Live Dashboard & Order Tracking Console

The central web interface syncs pending, packed, shipped, and completed orders from TikTok Shop in real-time. It groups and analyzes data, highlighting delayed orders or package anomalies. Staff can view detailed packing manifests, tracking IDs, recipient addresses, and buyer instructions.

2.2 Scan Parcel (The Packing Station Workflow)

This is the core execution interface operated at packing tables. It consists of the following steps:

- **Tracking Barcode Scan:** Operators scan the order's tracking number using a Zebra hardware scanner or the built-in webcam. The system queries the corresponding order lines in real-time.
- **Item Verification:** Staff scan each product barcode. The system verifies if the scanned barcode matches the designated TikTok Seller SKU and required quantities, flashing visual success/error cues.
- **Before-Pack Packing Proof:** Once verified, the camera captures an open-bucket photo of the correct items placed inside the packing bin. This is saved as quality proof.
- **After-Pack Shipping Proof:** The operator packs and seals the parcel. A final camera snapshot is taken showing the sealed courier bag with the shipping label stuck on top. The photo is uploaded to secure R2 Storage, and

the status changes to 'Packed'.

2.3 Artificial Intelligence Vision Guardrail

A major operational challenge is operator error or shortcuts—taking photos of empty boxes, unrelated materials, or loose sheets of paper. To solve this, the application integrates the Google Gemini 2.5 Flash API as a visual compliance gate:

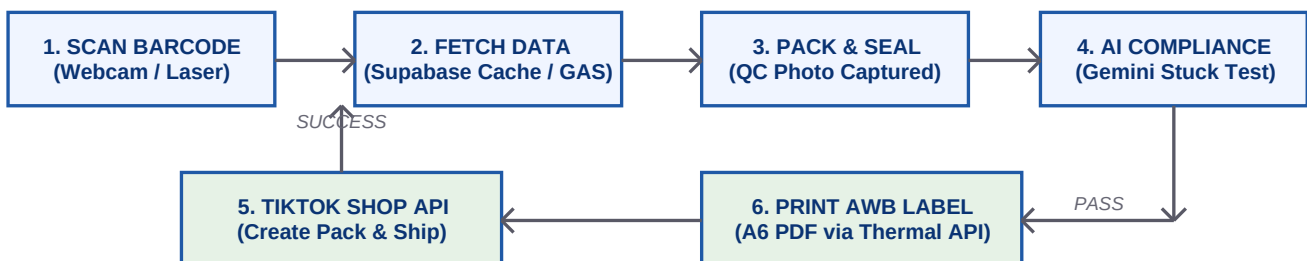
- **Loose Label Rejection:** When capturing the 'After-Pack Shipping Proof', the webcam frame is routed to the Cloudflare Worker, which calls the Gemini Vision model. The model extracts the tracking barcode and evaluates whether the label is physically stuck/adhered onto a box, plastic bag, bubble wrap, or bubble envelope. If the model returns 'stuck': false, the scan is rejected, warning the operator to apply the label before proceeding.

3. System Architecture & Integrated Data Flows

The fulfillment system uses a modern serverless architecture optimized for high speed, low latency, and military-grade security:

- **Client Side (Next.js 16):** Modern, full-bleed dashboard designed for desktop warehouse screens. Incorporates webcam capture streams, hardware barcode keyboard events, and automated thermal printing handlers.
- **Backend API Proxy (Cloudflare Workers):** A secure, low-latency API gateway routing requests. It handles authentication credentials, generates HMAC-SHA-256 signatures for TikTok Shop APIs, runs background token refreshes, and acts as a secure middleware for Gemini AI.
- **Database Caching Layer (Supabase SQL):** Retains order state cache to avoid hitting TikTok Shop API rate limits. Holds terminal registration IPs, user operator logs, and configuration values.
- **Secure Asset Storage (Cloudflare R2):** High-efficiency private object storage for packing photos, and document labels.

3.1 Integrated Data Flow Diagram



4. Technical API Endpoints & Request Helpers Reference

4.1 TikTok Shop Open API Signature Generation Helper

Every request targeting TikTok Shop Open Platform must include a valid query signature (sign) computed with HMAC-SHA-256. The app secret, endpoint path, query parameters, and JSON body are sorted and signed. Below is the secure signing implementation deployed inside the Cloudflare Worker backend:

```
async function generateTikTokSignature(
  path: string,
  params: Record<string, string>,
  appSecret: string,
  body: any = null
): Promise<string> {
  const sortedKeys = Object.keys(params).sort();
  let signString = appSecret + path;
  for (const key of sortedKeys) {
    signString += key + params[key];
  }
  if (body) {
    signString += typeof body === "string" ? body : JSON.stringify(body);
  }
  signString += appSecret;

  const encoder = new TextEncoder();
  const keyBuf = encoder.encode(appSecret);
  const dataBuf = encoder.encode(signString);

  const cryptoKey = await crypto.subtle.importKey(
    "raw", keyBuf, { name: "HMAC", hash: "SHA-256" }, false, ["sign"]
  );

  const signature = await crypto.subtle.sign("HMAC", cryptoKey, dataBuf);
  return Array.from(new Uint8Array(signature))
    .map(b => b.toString(16).padStart(2, "0"))
    .join("");
}
```

4.2 System-wide API Endpoint Mappings

The system exposes key routes to map operations from Next.js to the TikTok APIs:

1. Shop OAuth Connection Callback (/api/tiktok/auth/callback):

Exchanges authorization code for access/refresh tokens. Deploys resolveShopList() using GET /authorization/202309/shops to register authorized merchant accounts.

2. Sync Orders (/api/tiktok/orders):

Invokes POST /order/202309/orders/search to pull orders. Syncs tracking numbers, items, ciphers, and amounts. Restructures order lines into local cache datasets.

3. Create AWB and Pack Package (/api/tiktok/orders/create-awb):

If package_id does not exist, sends POST to /fulfillment/202309/packages to pack the order lines. Then calls POST /fulfillment/202309/packages/{package_id}/ship (handover_method: 'DROP_OFF') to schedule the carrier and generate the tracking ID.

4. Retrieve Printable Shipping Document (/api/tiktok/orders/print-awb):

Fetches documents from GET /fulfillment/202309/packages/{package_id}/shipping_documents (parameters: document_type: SHIPPING_LABEL_AND_PACKING_SLIP, size: A6, format: PDF). Returns the raw CDN URL to trigger silent client-side print jobs.

5. AI Label Scan & Stuck Verification (/api/tiktok/orders/ai-scan):

Webcam frames are forwarded here to call Gemini Vision APIs. Decodes barcodes and performs packaging compliance checks prior to submitting the 'Packed' status update.

5. TikTok Shop Open Platform Scopes Justification

To ensure the system works as intended, specific API scope packages are required. Each request is justified strictly by business/logistics requirements:

5.1 Scope: seller.shop.info (Shop Management)

Requirement: Used during OAuth authorization callback to verify connected shop identities. The system calls the GET /authorization/202309/shops API to retrieve the list of active merchant profiles, their assigned regions, and shop ciphers. The cipher is critical as a query parameter in all subsequent Order and Fulfillment endpoints. Without this scope, the system cannot verify authorization success or register shops.

5.2 Scope: seller.order.info (Order Management)

Requirement: Essential to fetch detailed order metadata from TikTok Shop (e.g., buyer info, tracking numbers, and line items). Warehouse terminals must compare physical products against the digital order to ensure packing correctness. The system calls POST /order/202309/orders/search and GET /order/202309/orders. Without this scope, the app cannot retrieve order lists, making error-free picking/packing technically impossible.

5.3 Scope: seller.fulfillment.basic (Fulfillment & Shipping)

Requirement: Critical to complete the order fulfillment loop. The packing terminal must update TikTok Shop once a package is physically packed (calling POST /fulfillment/202309/packages) and schedule shipment (calling POST /fulfillment/202309/packages/{package_id}/ship). Finally, the terminal must print the shipping label (AWB) directly at the station via the GET /fulfillment/202309/packages/{package_id}/shipping_documents endpoint. Without this scope, we cannot print labels or notify TikTok of dispatches, causing order cancellations and delivery delays.

6. Data Security and Privacy Standards

The system adheres to the highest level of security guidelines required by modern marketplace ecosystems:

- **Secure Credential Management:** The TikTok Shop App Key and Secret are stored as encrypted keys in a private database cache. These keys are never exposed to the client or browser.
- **Token Life-cycle Controls:** Access tokens are held securely on the Cloudflare Workers middleware. An automated helper ensures tokens are refreshed dynamically, avoiding data leaks or connection loss.
- **R2 Secure Media Vault:** Quality control photos (Before/After Pack Proof) are written directly to private Cloudflare R2 Buckets, secured behind temporary pre-signed links. Media is strictly deleted upon order completion or if a transaction is canceled.
- **Audit Logs:** Every single action performed—from OAuth configuration to printing labels—is tracked with timestamps, operators' IDs, and terminal IPs in a secure audit log sheet.